

Training and Backpropagation in NMT: 2-layer Encoder & 4-layer Decoder (LSTM, 20 units)

Dr. Muhammad Usama and ChatGPT

Overview

This document describes the forward pass, loss, and backpropagation for a sequence-to-sequence NMT model with:

- Encoder: 2 stacked LSTM layers (parameters $W_1^{(e)}, W_2^{(e)}$), 20 units per layer.
- Decoder: 4 stacked LSTM layers (parameters $W_1^{(d)}, W_2^{(d)}, W_3^{(d)}, W_4^{(d)}$), 20 units per layer.
- Embedding size: E . Source length T_x , target length T_y .
- Loss: token-level cross-entropy summed over decoder timesteps.

1. Forward pass

- **Encoder (layered, left-to-right):**

$$(\mathbf{h}_{1,t}^{(e)}, \mathbf{c}_{1,t}^{(e)}) = \text{LSTM}_1^{(e)}(\mathbf{x}_t, (\mathbf{h}_{1,t-1}^{(e)}, \mathbf{c}_{1,t-1}^{(e)}))$$

$$(\mathbf{h}_{2,t}^{(e)}, \mathbf{c}_{2,t}^{(e)}) = \text{LSTM}_2^{(e)}(\mathbf{h}_{1,t}^{(e)}, (\mathbf{h}_{2,t-1}^{(e)}, \mathbf{c}_{2,t-1}^{(e)}))$$

Final encoder state: $(\mathbf{h}_{2,T_x}^{(e)}, \mathbf{c}_{2,T_x}^{(e)})$.

- **Decoder initialization:** decoder initial states are initialized from encoder final states (identity or learned projection):

$$\mathbf{h}_{\ell,0}^{(d)} = \text{init}_\ell(\mathbf{h}_{2,T_x}^{(e)}), \quad \mathbf{c}_{\ell,0}^{(d)} = \text{init}_\ell(\mathbf{c}_{2,T_x}^{(e)})$$

- **Decoder (4 layers) with teacher forcing** for target steps $t' = 1 \dots T_y$:

$$(\mathbf{h}_{1,t'}^{(d)}, \mathbf{c}_{1,t'}^{(d)}) = \text{LSTM}_1^{(d)}(\mathbf{y}_{t'-1}^{emb}, (\mathbf{h}_{1,t'-1}^{(d)}, \mathbf{c}_{1,t'-1}^{(d)}))$$

$$(\mathbf{h}_{\ell,t'}^{(d)}, \mathbf{c}_{\ell,t'}^{(d)}) = \text{LSTM}_\ell^{(d)}(\mathbf{h}_{\ell-1,t'}^{(d)}, (\mathbf{h}_{\ell,t'-1}^{(d)}, \mathbf{c}_{\ell,t'-1}^{(d)})) \quad (\ell = 2, 3, 4)$$

Output logits are produced from the top decoder layer $\mathbf{h}_{4,t'}^{(d)}$.

- **Loss:**

$$\mathcal{L} = - \sum_{t'=1}^{T_y} \log P(y_{t'}^{true} \mid y_{<t'}^{true}, x)$$

2. Backpropagation (gradient flow)

Backpropagation follows the computation graph in reverse:

A. Decoder parameter gradients. For each decoder layer ℓ :

$$\frac{\partial \mathcal{L}}{\partial W_{\ell}^{(d)}} = \sum_{t'=1}^{T_y} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{\ell,t'}^{(d)}} \cdot \frac{\partial \mathbf{h}_{\ell,t'}^{(d)}}{\partial W_{\ell}^{(d)}}$$

Gradients accumulate across timesteps (BPTT) and across upward connections within each timestep (layer $\ell + 1 \rightarrow \ell$).

B. Gradients flowing into encoder. Gradients that reach decoder initial states flow back into encoder final states and then back through encoder time/layers:

$$\frac{\partial \mathcal{L}}{\partial W_{\ell}^{(e)}} = \sum_{t=1}^{T_x} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{\ell,t}^{(e)}} \cdot \frac{\partial \mathbf{h}_{\ell,t}^{(e)}}{\partial W_{\ell}^{(e)}}$$

where $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{\ell,t}^{(e)}}$ includes contributions from decoder initialization and from encoder recurrence.

C. Key intuition

- $W^{(d)}$ do *not* directly change $W^{(e)}$. Instead, the decoder architecture and parameters shape the error signal (the quantity $\partial \mathcal{L} / \partial h$) that is propagated back into the encoder.
- Each parameter matrix receives gradients only from the nodes it influences in the forward pass; the chain rule decomposes the gradient accordingly.

3. Practical tips

- Use **teacher forcing** during training (optionally with scheduled sampling).
- Apply **gradient clipping** (global norm) to prevent exploding gradients.
- Use **Adam** or similar adaptive optimizer with LR scheduling.
- Mask loss for padded tokens in batches.
- Consider an **attention** mechanism for better gradient paths and long sentences (decoder reads all encoder hidden states, so gradients flow to encoder at every source timestep).
- Use dropout between LSTM layers for regularization.